Experiment No : 9

Title: Represent graphs and generate minimum spanning trees using Prim's algorithm.

Name of Student: Shamiksha Chaudhari
Class: SY **Div:** SY1 **PRN No:** B25ET1005 **Batch:** A1

Program:

```
#include <stdio.h>
#include <limits.h>

#define V 5   // Number of vertices in the graph

// Function to find the vertex with the minimum key value
int minKey(int key[], int mstSet[]) {
    int min = INT_MAX, min_index;

    for (int v = 0; v < V; v++) {
        if (mstSet[v] == 0 && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}

// Function to print the constructed MST
void printMST(int parent[], int graph[V][V]) {
    printf("Edge   Weight\n");
    for (int i = 1; i < V; i++)
        printf("%d - %d    %d \n", parent[i], i, graph[i][parent[i]]);
}

// Function to construct and print MST for a graph represented using adjacency matrix
void primMST(int graph[V][V]) {
```

```
    int parent[V];   // To store constructed MST
    int key[V];      // Key values used to pick minimum weight edge
    int mstSet[V];   // To represent set of vertices not yet included in MST

    // Initialize all keys as infinite
    for (int i = 0; i < V; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }

    // First vertex is always the root of MST
    key[0] = 0;
    parent[0] = -1; // First node is root

    // MST will have V vertices
    for (int count = 0; count < V - 1; count++) {
        int u = minKey(key, mstSet);  // Pick min key vertex not yet in MST
        mstSet[u] = 1;

        // Update key value and parent index of the adjacent vertices
        for (int v = 0; v < V; v++) {
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    printMST(parent, graph);
}

// Driver code
int main() {
    // Example graph represented as adjacency matrix
    int graph[V][V] = {
        {0, 2, 0, 6, 0},
        {2, 0, 3, 8, 5},
```

```
        {0, 3, 0, 0, 7},
        {6, 8, 0, 0, 9},
        {0, 5, 7, 9, 0}
    };

    primMST(graph);

    return 0;
}
```

Simpler Version

```
// Online C compiler (editor)
// Write and run C online using this editor.

#include <stdio.h>

#define V 5

int main()
{
    int graph[V][V] = {
        {0, 2, 0, 6, 0},
        {2, 0, 3, 8, 5},
        {0, 3, 0, 0, 7},
        {6, 8, 0, 0, 9},
        {0, 5, 7, 9, 0}
    };

    int visited[V] = {0};
    int edges = 0;
    int min, u, v, total = 0;

    visited[0] = 1;  // Start from vertex 0

    printf("Edge  Weight\n");

    while (edges < V - 1)
    {
```

```
        min = 999;

        for (int i = 0; i < V; i++)
        {
            if (visited[i] == 1)
            {
                for (int j = 0; j < V; j++)
                {
                    if (visited[j] == 0 && graph[i][j] != 0)
                    {
                        if (graph[i][j] < min)
                        {
                            min = graph[i][j];
                            u = i;
                            v = j;
                        }
                    }
                }
            }
        }

        printf("%d - %d   %d\n", u, v, min);

        visited[v] = 1;
        total = total + min;
        edges++;
    }

    printf("Total MST Weight = %d\n", total);

    return 0;
}
```

OUTPUT:

```
Enter number of vertices: 3
Enter adjacency matrix:

2
3
4
5
6
7
8
9
1

Edges in Minimum Spanning Tree:
1 - 2 : 3
1 - 3 : 4
Total cost of MST = 7
```